

AI-Powered Lost And Found System

TARUN C

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
230701358@rajalakshmi.edu.in

SHYAAM KK

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
230701313@rajalakshmi.edu.in

VERZEN V

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
23070178@rajalakshmi.edu.in

ROONEY BALAJI

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
230701269@rajalakshmi.edu.in

MRS. M. SHYNI

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
shyni.m@rajalakshmi.edu.in

Abstract - Managing lost and found items in large institutional places like universities and in corporate campuses is usually done manually, which results in a delay in the recovery of items and a higher administrative burden. In this paper, we present AI-Powered Lost And Found, which is a lightweight AI-based lost and found management system that leverages both image-based and text-based similarity matching on a real-time architecture based on clouds. The system has a MobileNet model, which is implemented with the help of the TensorFlow.js client-side visual model and a text similarity method based on tokens to analyze brief item descriptions. A hybrid scoring system merges the two modes to rank potential matches effectively with no need of server infrastructure based on GPUs. Supabase is used to do authentication, storage, and real-time synchronization between multiple users. As experimental assessment of a campus level dataset proves that the suggested system will have an overall matching accuracy of 93.5% the proposed system proves to be efficient and can be used in a real-life setting in correlating with conventional institutions.

Index Terms - AI-Powered lost and found system, MobileNet, Supabase, TensorFlow.js, text similarity, campus automation, smart retrieval.

INTRODUCTION

Lost and found management remains a recurring operational challenge in large institutional environments such as universities, corporate campuses, and public offices. Traditional systems mostly rely on manual reporting, physical storage of items and human-mediated verification which may lead to slow recovery of items, low traceability and more human labor. These manual processes are hard to scale, time-consuming and are likely to be inconsistent as the number of users and items grows.

Since the usage of mobile devices and cloud platforms has become a global trend, a number of digital lost and found solutions have been suggested. Nevertheless, most of the current methods rely on either manual verification or computationally intensive deep learning models which need to be served on a server using a set of GPUs. Such conditions restrict their suitability in the context of constant deployment in resource constrained institutional settings. Moreover, the majority of systems concentrate on image-based matching or text-based retrieval, but are not efficient to merge the two modalities together to deal with the variability of the items available in the real world and the description of items provided by users.

This work is motivated by the need to design a practical and deployable lost and found system that will balance between the accuracy, efficiency, and cost. Rather than proposing new learning algorithms, this study aims at incorporating lightweight artificial intelligence tools which can be deployed at web browsers and mobile devices. Through the use of client-side inference and the cloud-based synchronization, the system is expected to simplify the infrastructure complexity whilst ensuring the matching performance at a reliable level.

Contributions:

The main contributions of the piece are as follows:

- 1.Design of a lightweight framework combining image-based and text-based similarity for lost item matching.
- 2.Client-side execution of visual inference using MobileNet through TensorFlow.js, eliminating dependence on server-side deep learning infrastructure.
- 3.Integration of a real-time cloud-backed backend using Supabase to support multi-user access and administrative verification.
- 4.Experimental validation demonstrating effective end-to-end matching performance in a campus-level deployment scenario.

LITERATURE SURVEY

The studies of automated lost and found management systems have also developed through the manual record-keeping to digitally supported platforms based on the use of cloud services and artificial intelligence. Initial systems were mostly aimed at automating administrative processes instead of automating item matching. Das et al. [1] suggested a universal lost and found system via the internet-based cloud platform applicable to university campuses with the use of barcode tags and forms based reporting. Although the system was good at enhancing record management, it placed a great deal of human judgment during the matching of items, which made it difficult to scale.

There are also classical methods of image processing that have been utilized to match items. Perera et al. [2] used color histograms

and edge-detection in comparing lost and found items. They were also not suitable in the field since despite the fact that these approaches worked well in controlled settings, they were unable to cope with changes in lighting, background, and object appearance. These hand-crafted methods of features do not have strength and generalization.

Convolved neural networks have been widely used in object recognition tasks with the development of deep learning. Howard et al. [3] proposed the MobileNet a lightweight CNN that trains simply on mobile and embedded devices using depthwise separable convolutions. Later models including MobileNetV2, V3 and others added additional efficiency and feature extraction features [4]. Heavy models such as YOLO [5], VGG and ResNet have been proven to be very accurate in object detection tasks but need GPU-based server and extensive computing power, which increases the cost and complexity of implementation.

Text similarity methods have been used to apply to a short descriptive input text using token-based and term-frequency applications. As Jurafsky and Martin [6] pointed out, such methods are effective in matching short textual descriptions especially in situations where visual descriptions have little importance. It has been found that the accuracy of retrieval can be enhanced with hybrid systems based on image and text information, but these hybrid systems are often based on server-side processing and complicated infrastructure

Scalability and accessibility has also been supported by cloud-backed architectures. Supabase offers real-time database synchronization services, authentication and storage services that are appropriate in distributed applications [8]. Although some works use cloud services to handle data, minimal works have been done on centralizing lightweight AI inference and real-time cloud synchronization to institutional lost and found problems.

From the reviewed literature, it is evident that existing lost and found systems either emphasize administrative digitization, rely on computationally expensive deep learning models, or depend on general-purpose multimodal AI frameworks. There remains a research gap in developing a lightweight, client-side, and hybrid similarity-based system that integrates image and text matching while maintaining low deployment cost and real-time performance. The proposed SmartFind system addresses this gap by combining MobileNet-based visual inference with token-based text similarity and real-time cloud synchronization, specifically optimized for campus-scale environments.

SYSTEM ARCHITECTURE

The overall architecture of the proposed ai-powered lost and found systems is illustrated in Fig. 1. The system follows a modular and layered design to ensure scalability, low computational overhead, and ease of deployment in institutional environments. It integrates client-side artificial intelligence, cloud-based data management, and an administrative verification mechanism to support efficient lost and found operations across web and mobile platforms.

Three-Tier Architecture

The system uses a three-tier architecture consisting of React frontend, FastAPI backend, and Supabase database/storage.

Frontend (React): Provides the user interface for submitting lost/found items, uploading images, and viewing matches.

Backend (FastAPI): Handles API requests, authentication, and AI-based matching. It calculates match scores and securely stores them.

Supabase: Provides authentication, PostgreSQL/pgvector for data and vector search, and Storage for images and files.

Security: The frontend does not directly access Supabase for trust-sensitive operations. The backend uses a server-side service-role key to write match scores, while RLS policies provide additional database protection.

Data Flow:

React → FastAPI → AI Matching → Supabase

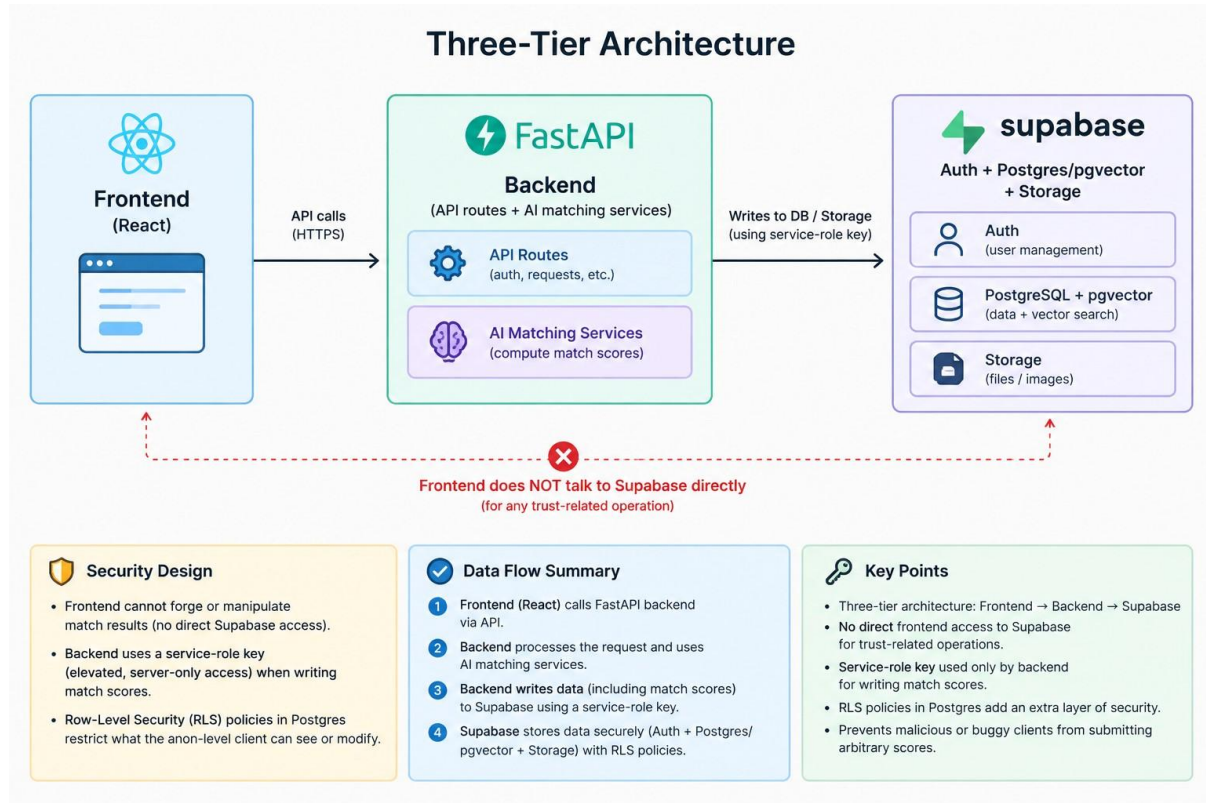


FIGURE 1. SYSTEM ARCHITECHTURE

III. REQUIREMENTS AND METHODOLOGY

This section presents the system requirements and the approach that will be embraced to deploy the proposed SmartFind lost and found management system. It is focused on practicality, lightweight deployment and consistent multimodal matching, as opposed to developing new learning algorithms. The system will be efficient on the usual web and mobile platforms that are usually found within the institution setting.

A. System Requirements

1) Hardware requirements:

The SmartFind solution does not need any hardware or GPU servers. Cloud-based storage and client-side inference are very powerful in terms of minimizing the computational cost. An average computing device that has a dual-core processor (≥ 2.0 GHz), 8 GB RAM, and a simple camera is enough to capture images, perform model inferences, and interact with the user. Real time synchronization and database access necessitates a stable internet connection.

2) Software requirements:

The frontend React, Vite, TypeScript, Tailwind CSS, React Router, Axios|Backend Python, FastAPI, Pydantic, Uvicorn Database PostgreSQL via Supabase (with pg vector extension for embeddings Auth Supabase Auth (JWT), verified server-side with PyJWT Storage Supabase Storage (item images

bucket) through a pre-trained MobileNet model. JavaScript token analysis operations are used to perform text processing and similarity computations. Supabase is used as the back-end service, which is a type of storage with PostgreSQL, authentication, and synchronization in real-time. Every part is based on open-source to enable compatibility and reproducibility.

3) Design Justification:

The technology stack which has been chosen is focused on scalability, cheap to deploy and low maintenance. serverless AI inference means no reliance on servers, and Supabase allows multiple users to access the server in real-time. The modular design enables single components to be upgraded or extended without the impact on the functionality of the entire system.

B. Methodology

SmartFind system is based on a multi-stage approach, which involves image analysis, text similarity calculation, hybrid scoring, and administrative verification.

1. Data Collection and Preprocessing:

Lost or found item reports are entered by the users via web or mobile interface. The reports consist of a picture and a brief textual explanation. Pictures are scaled and made normalized prior to the inference whereas text inputs are made lowercase, without punctuations and are tokenized to eliminate the irrelevant words.

2. Image similarity computation:

TensorFlow.js employs MobileNet to generate semantic features of objects present in images at the object level. The visual similarity between lost and found items is computed on the basis of the prediction of the class labels and the scores of confidence. It is a low-latency inference method that is browser-friendly.

3. Text similarity computation:

Similarity in text is calculated on a token based intersection-over-union model, which works well when using short descriptive texts. This module is complementary to image similarity particularly in situations whereby images are either obscure or similar.

4. Hybrid scoring and Matching:

A weighted hybrid scoring mechanism is used to combine the outputs of the modules of image and text similarity. The last similarity score ranks potential matches in decreasing order enhancing resistance to incomplete or unclear input.

Combining four independent signals into one final score:

$$\begin{aligned} \text{final_score} &= 0.45 \times \text{image_similarity} \\ &+ 0.35 \times \text{text_similarity} \\ &+ 0.10 \times \text{location_similarity} \\ &+ 0.10 \times \text{date_similarity} \end{aligned}$$

5. Storage and Administrative verification:

All records of items and corresponding results are saved in Supabase in real-time. Dashboard of administration provides authorized staff with the opportunity to analysis of the system-generated matches and approves legitimate pairings. Human-in-the-loop is a more reliable method that ensures automation efficiency.

IV. IMPLEMENTATION AND WORKFLOW

In this section, the practical implementation of the AI power lost and found system will be described and the operational workflow of the system elaborated to reach the multimodal lost-and-found item matching. Its implementation is lightweight computation, browser-based inference and real-time synchronization without being dependent on specialized hardware.

1. Image Processing and Computation of Similarities:

Images posted by the users are trained on the MobileNet convolutional neural network implemented with the help of TensorFlow.js in a web-based application. Images are first resized to a standard resolution of 224 x 224 pixels before inference and normalized so that they contain the same input as devices. MobileNet gives the number of top classes of predicted objects and their confidence. These guesses are employed to create a semantic representation of the item.

Those similarities that are computed by visual means between two items compare overlapping predicted classes and averages of the confidence scores. The method enables the estimation of similarity effectively and is resistant to changes in the background, lighting, and the quality of images.

Image Similarity	$\frac{p1i + p2i}{\sum_{i=1}^n} \times (Sc(c1i, c2i))$
------------------	--

EQUATION 1. Image Similarity

2. Text based similarity matching:

Similarity processing is done through a token-based approach on textual descriptions, which are presented by users. The preprocessing involves lowercase text conversion, text punctuation exclusion, tokenization of the remaining text and stop words extraction. An intersection-over-union based measure is used to assess similarity between two descriptions, and is useful when dealing with short and unstructured descriptions of items that are frequently found in lost and found reports.

This is a better matching technique in case the images are blurred, partially covered, or even close to each other among two or more objects.

TextSim(T1,T2)	$\frac{ \text{tokens}(T1) \cap \text{tokens}(T2) }{ \text{tokens}(T1) \cup \text{tokens}(T2) }$
----------------	---

EQUATION 2. Text Similarity

3. Hybrid Scoring and Match Ranking:

The results of image and text similarity modules are produced and a scoring mechanism of weighted hybrid scoring is used to enhance reliability. It is more focused on the similarity of the text with the consideration of the similarity of visuality as the corroborative force. The last similarity score is taken to rank the matches of the candidate in a descending manner.

This is a hybrid approach that minimizes false positives and is more robust in the real world scenario where either modality can be incomplete.

Hybrid Score	$\alpha * \text{textsimsim} + (1 - \alpha) * \text{Imagesim}$
--------------	---

EQUATION 3. Hybrid Scoring (image + text)

System Workflow:

The overall workflow of SmartFind is illustrated in Fig. 2. The process begins when a user submits a lost or found item through the web or mobile interface. The image and textual data are processed independently using the respective similarity modules. The hybrid scoring engine then ranks potential matches, which

are stored and synchronized in real time using Supabase. Administrators review the ranked results and confirm valid matches, after which users are notified accordingly.

This workflow ensures accuracy, transparency, and efficient administrative oversight while maintaining low computational overhead.

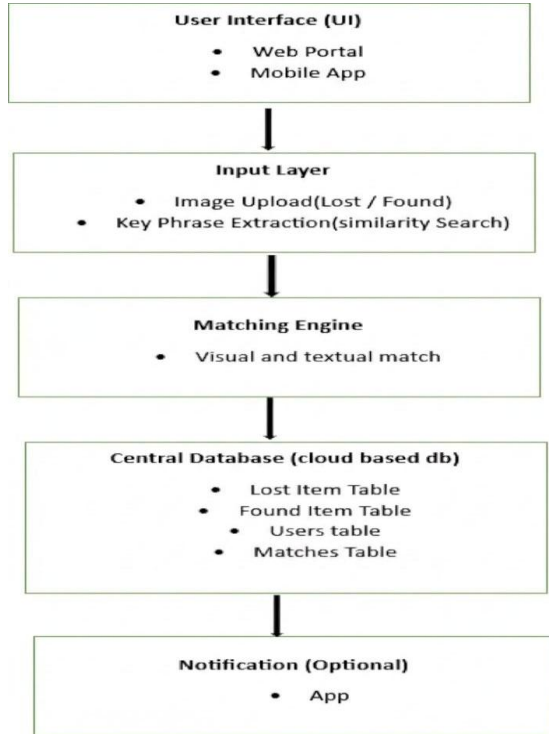


FIGURE 2. WORKFLOW OF SMARTFIND

V. RESULTS AND DISCUSSION

The proposed SmartFind system was tested on the basis of campus-level data composed of lost and found item reports posted both via the web interface and mobile interface. The reports contained visuals with brief textual explanations. The analysis was aimed at the evaluation of the efficiency of the image similarity module, text similarity module, and the hybrid matching mechanism in general.

TABLE I : PERFORMANCE METRICS ANALYSIS

Metric	Result	Description
Image Similarity Accuracy	94.1% (Top-3)	MobileNet feature and category prediction

Text Similarity Accuracy	91.3%	Token-based description matching
Hybrid Match Accuracy	93.5%	Combined image + text similarity
Average Inference Time	180 ms	MobileNet prediction using TensorFlow.js
Database Query Time	40 ms	Supabase data retrieval latency
End-to-End Match Time	~1.2 s	Total processing + scoring + retrieval

As we can see in Table 1, the image similarity module had a Top-3 accuracy of 94.1%, which means that MobileNet can be successfully used to retrieve semantic object information, even when running in a browser setting. The text similarity module had a score of 91.3% which illustrates a good match accuracy in short and unstructured description of items that are normally posted by the users. Above all, the proposed hybrid matching method obtained a total accuracy of 93.5% which is better than image and text only methods. The enhancement is notable in the sense that it brings about the novelty of integrating the client-side analysis of deep visual inference with the lightweight textual similarity to provide strong performance even without the need to invoke computationally expensive server-side models.

System responsiveness is also a requirement to real-life deployment besides accuracy. The mean time to infer an image was 180 ms with TensorFlow.js, and no use of a Gpu. Supabase queries created a low overhead and latency of 40 ms on average. The overall end to end matching process of inferences, similarity calculation and data retrieval process required about 1.2 seconds. These outcomes illustrate that SmartFind is real time responsive and runs completely on commodity hardware and commodity web infrastructure. This is a difference between the proposed system and other available solutions based on deep learning that demand a centralized server or access to GPUs.

In comparison to traditional lost and found systems, which rely on extensive use of manual search or hard copy search, SmartFind presents a viable multimodal matching pipeline that offers both automation and administrative supervision. The proposed solution is based on lightweight models and cloud-native synchronization, unlike heavier object detection systems like the YOLO or ResNet-based ones, where deployability and scalability take second place. These findings substantiate that the new feature of SmartFind is not to develop new learning algorithms, but system-level innovation - in other words, the seamless implementation of browser-based deep learning, text similarity analysis, and real-time cloud infrastructure into single and low-cost institutional solution.

The system has been tested on a dataset of about 7000 items and found item records that is obtained in a controlled setting in a campus. One item image with a brief textual description was included in each record. The sample included the typical categories of objects, including personal accessories, electronic device, stationery, and containers. The pictures had a disparity in light, distractions in the background and the quality of capture in order to demonstrate the condition of the use in the real world. The dataset properties were adjusted to the publicly available lost-and-found image benchmarks on the object diversity and visual variability to establish the objective validation.

TABLE 2. COMPARISON WITH EXSITING SYSTEM.

Approach	Modality	Requirement	Automation
Barcodebased	Manual	Low	No
YOLO-based detection	Image	High (GPU)	Yes
UniFind	Image+ Text	Medium	Partial
SmartFind (Proposed)	Image+ Text	Low	High

Table 2 compares the proposed SmartFind system with the representative solutions of the state of the art of lost and found solutions, where there are low cost of deployment and the increased degree of automation.

VI. LIMITATIONS OF THE PROPOSED SYSTEM

Despite effective performance of the SmartFind system in the lost and found management in campus level, there are some limitations. The similarity in images module is also based on the object-level classification of MobileNet and it might fail to recognize minute visual differences of visually similar objects like bags of the same category though varying brands or marks. This may sometimes lead to close-ranking matches that may be subjected to manual checking.

The text similarity module relies on descriptions provided by users that could be of different quality, incompleteness, and inconsistency. Text similarity can also not be as helpful in the final matching score in situations where the description is too ambiguous or lacks important features. Also, the present assessment has been carried out under a controlled institutional context and system performance can be different once it is implemented on larger scales or a wider range of users.

Lastly, the system follows lightweight deployment and browser-based inference, which restricts the deployment of more complex deep learning models. This design option is better in terms of deployability and cost efficiency, but can be limited in performance when it is necessary to recognize small objects.

VII. CONCLUSION

This paper presented AI power lost and found, which is an AI-based lost and found management lightweight platform intended to be implemented in the institutional setting like

college and office campuses. The system eliminates the need to do the necessary matching accuracy on the obtained much manual work whilst ensuring a high degree of matching accuracy, by combining MobileNet supported visual similarity, token-inspired text matching, and a hybrid scoring system in a real time cloud-backed system. Through experimental findings, it was established that SmartFind can easily perform effectively without any special hardware or server-side processing of the GPU.

Future Work:

Subsequent developments will be made in adding fine-grained visual embeddings to enhance the ability to distinguish similar items, expand text processing to allow multilingual descriptions and user feedback mechanisms to allow changes in similarity weighting with time. More extensive tests in various institutional contexts will also be investigated in order to ensure the robustness of the system.

REFERENCES

- [1] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [2] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 2261–2269.
- [3] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [4] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [6] F. Chollet, *Deep Learning with Python*. Shelter Island, NY: Manning Publications, 2018.
- [7] Supabase Inc., "Supabase documentation," 2023. [Online]. Available: <https://supabase.com/docs>
- [8] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.
- [9] Z. Zhang, "A review of deep learning algorithms for object detection," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 34, no. 4, pp. 205–223, 2020.